Python
Project Track

Build first. Learn the idea while you build.

Stage 1: Interactive programs
Stage 2: Programs that remember data
Stage 3: Organised programs with functions
Stage 4: Your first APIs
Stage 5: Your own capstone project

How This Track Works

Every project here is something that runs and does a real job. You do not study a concept first and then try to use it. You start building, and the concept shows up when you need it.

Every project comes in a pair

Type	What you get	How to work
Guided	A goal, numbered steps, a sample output, and hints.	Follow the steps. Try each step on your own before opening a hint.
Unguided	A brief only. No steps, no hints, no starter code.	Plan it yourself. Write the steps as comments first, then fill in the code.

The rhythm for each project

1. Read the goal and say it in your own words.
2. Build the smallest working version first. Run it.
3. Add one feature at a time. Run it again after each one.
4. When it works, read your code once from top to bottom and fix anything messy.
5. Write one line in your Build Log (last page).

When you get stuck (you will, and that is normal)

The 10-minute rule
1. Read the last line of the error message. 2. Shrink the task to the smallest next step, for example print one value only. 3. Work on that for 10 more minutes before you look for help. Getting stuck is not a sign that you are not good at this. It is the part of programming where you actually learn.

The projects in this track

Stage	Guided	Unguided
1. Interactive programs	Bill Splitter, Quiz Game	Monthly Budget Planner, Password Checker
2. Remembering data	Contact Book, Expense Tracker	Class Marks Analyser
3. Functions	Gradebook	Library Manager
4. APIs	Weather Checker, Country Info Card	City Comparison Tool
5. Capstone	Choose one of three	Your own build

STAGE 1

Interactive Programs

What you build in this stage
Programs that ask questions, calculate and make decisions. Ideas you will meet: variables, input(), type conversion, operators, f-strings, if / elif / else, while and for loops.

GUIDED 1.1 Bill Splitter

Goal: variables, input, type conversion, operators, f-strings.

You will use: input(), float(), int(), round(), f-strings

1. Ask for the **total bill**, the **tip percentage** and the **number of people**.
2. Convert the inputs to the right types.
3. Calculate the tip amount, the total with tip and the amount **each person** pays.
4. Print a neat summary with every amount rounded to 2 decimal places.

```
Total bill: 4500
Tip %: 10
People: 3
-------------------------
Tip:          450.00
Total:       4950.00
Each person: 1650.00
```

Hints (try without them first)
input() gives text. Use float() for money and int() for people. 10% of a bill is bill * tip / 100.

GUIDED 1.2 Quiz Game

Goal: if / else, loops, a score counter.

You will use: a list of questions, a for loop, if / else, a counter

1. Make a list of 5 questions. Each one is a small list of two items: the question and the answer, for example `["Capital of Kenya?", "nairobi"]`.
2. Create a **score** that starts at 0.
3. Use a for loop to ask each question. Compare the answer in lower case so "Nairobi" also counts.
4. Add 1 to the score for each correct answer and print "Correct!" or the right answer.
5. At the end print the score, for example "You got 4 out of 5", with a message that depends on the score.
6. *Stretch:* let the player retry a wrong answer once for half a point.

Hints (try without them first)
Use .lower() on the answer. Use question[0] for the question and question[1] for the answer.

UNGUIDED 1.3 Monthly Budget Planner

Brief only. No steps, no hints, no starter code.

Brief
Ask the user for their monthly income and at least 4 expenses (for example rent, food, transport, airtime). Show the total spent, the money left, and the percentage of income each expense takes. If the user spends more than they earn, print a clear warning. If they save at least 20% of income, print an encouraging message.

Must include: variables, input() with conversion, if / elif / else, a loop for the expenses, round().

Extension: ask for a savings goal and say how many months it would take to reach it.

Explain it (be ready to answer these)
Why do you need float() before you do maths with input()? What would happen if the income is 0?

UNGUIDED 1.4 Password Strength Checker

Brief only. No steps, no hints, no starter code.

Brief
Ask the user for a password. Check its length, whether it contains a number, whether it contains a capital letter and whether it contains a symbol. Give it a score and say Weak, Medium or Strong. Keep asking until the user enters a Strong password.

Must include: a while loop, a for loop over the characters of a string, if statements, and string methods such as isdigit() and isupper().

Extension: tell the user exactly which rules are missing.

Explain it (be ready to answer these)
How do you loop over a string? Why is a while loop better than a for loop for 'keep asking until...'?

STAGE 2

Programs That Remember Data

What you build in this stage
Programs that store many pieces of information at once. Ideas you will meet: lists, dictionaries, a list of dictionaries, a menu loop, searching.

GUIDED 2.1 Contact Book

Goal: dictionaries, menu loop, searching.

You will use: a dictionary, while True, break, if / elif, in

1. Create an empty dictionary called **contacts**. A name is the key and a phone number is the value.
2. Show a menu: 1 Add, 2 Search, 3 Show all, 4 Delete, 5 Exit. Repeat it in a while loop until the user chooses 5.
3. Option 1: ask for a name and a number and save them.
4. Option 2: ask for a name. Print the number, or "Not found".
5. Option 3: print every contact with a loop using .items().
6. Option 4: delete a contact if it exists.
7. Handle a wrong menu choice with a friendly message.

```
1. Add  2. Search  3. Show all  4. Delete  5. Exit
Choose: 1
Name: Amina
Number: 0712345678
Saved!
```

Hints (try without them first)
contacts[name] = number adds an entry. name in contacts checks if it exists. del contacts[name] removes it.

GUIDED 2.2 Expense Tracker

Goal: a list of dictionaries, totals by category.

You will use: a list of dictionaries, a for loop, sum(), a second dictionary for totals

1. Create an empty list called **expenses**.
2. Let the user add expenses until they type "done". Each expense is a dictionary, for example `{"item": "lunch", "amount": 300, "category": "food"}`.
3. Print all expenses in a neat list.
4. Print the **total** spent and the **biggest** expense.
5. Build a dictionary of **totals per category** by looping through the list. Print it.
6. *Stretch:* print the category the user spends the most on.

Hints (try without them first)
Check if the category is already in your totals dictionary. If it is, add to it. If not, set it first.

UNGUIDED 2.3 Class Marks Analyser

Brief only. No steps, no hints, no starter code.

Brief
Store at least 6 students as a list of dictionaries. Each student has a name, a class and a list of three marks. Print each student's average, then show the top student, the class average and the students whose average is below 50. Let the user type a name to look up one student's full record.

Must include: a list of dictionaries, loops, sum() and len(), searching by name.

Extension: sort the students from highest to lowest average and print a ranked list.

Explain it (be ready to answer these)
When do you choose a dictionary and when a list? What happens if the name the user types does not exist?

STAGE 3

Organised Programs with Functions

What you build in this stage
Splitting a program into small, named parts that each do one job. Ideas you will meet: def, parameters, return, default values, main().

GUIDED 3.1 Gradebook with Functions

Goal: def, parameters, return, main().

You will use: functions with return values, a default parameter, a main() function

1. Write `calculate_average(marks)` that takes a list of marks and **returns** the average.
2. Write `get_grade(average)` that returns "A" to "E" using if / elif.
3. Write `is_passing(average, pass_mark=50)` that returns True or False. Notice the default value.
4. Write `print_report(name, marks)` that calls the three functions above and prints a neat report.
5. Write `main()` that asks for a name and 3 marks, and calls print_report().
6. Call main() at the bottom of the file.

```
Report for Amina
Marks: [78, 65, 82]
Average: 75.0   Grade: B   Result: PASS
```

Hints (try without them first)
A function that prints returns nothing. A function that returns hands a value back to the code that called it.

UNGUIDED 3.2 Library Manager

Brief only. No steps, no hints, no starter code.

Brief
Build a menu-driven program for a small library. Keep books in a list of dictionaries (title, author, available as True or False). The user can add a book, borrow a book, return a book, list all books and list only the available ones. Borrowing a book that is already out, or a title that does not exist, must show a clear message.

Must include: at least 6 functions, a main() function with the menu loop, a list of dictionaries, input validation.

Extension: record who borrowed each book and show the names in the list.

Explain it (be ready to answer these)
Why is each function better doing one job? What is the difference between a parameter and an argument?

STAGE 4

Your First APIs

What you build in this stage
An API lets your program ask another program for information over the internet. The answer usually comes back as **JSON**, which looks exactly like the dictionaries and lists you already know. Nothing here is new. You are using Stages 1 to 3 on live data.

Before you start
Install the tool once in your terminal: `pip install requests`. Then remember the pattern: **requests.get(url)** sends the question, **.json()** turns the answer into a Python dictionary, and then you use your normal dictionary skills to pick out what you need.

GUIDED 4.1 Weather Checker

Goal: calling an API and reading the result.

You will use: the requests library, a dictionary, f-strings, a function

Start by running this code. Look at what is printed before you change anything.

```
import requests

url = "https://api.open-meteo.com/v1/forecast"
params = {
    "latitude": -1.29,
    "longitude": 36.82,
    "current_weather": True,
}
response = requests.get(url, params=params)
data = response.json()
print(data)
```

1. Find the part of the output called **current_weather**. It is a dictionary inside a dictionary.
2. Print only the temperature: `data["current_weather"]["temperature"]`. Then print the wind speed the same way.
3. Print a friendly sentence with both values in an f-string.
4. Put the code inside a function `get_weather(latitude, longitude)` that **returns** the temperature and wind speed.
5. Look up another city's latitude and longitude and call your function with them.
6. Handle problems: if the request fails, print a friendly message instead of crashing. Wrap the request in try / except `requests.exceptions.RequestException`.

Hints (try without them first)
If you see an error, check the spelling of the keys. Print data first and read it carefully. A key error usually means a small spelling difference.

GUIDED 4.2 Country Info Card

Goal: reading nested JSON, lists and dictionaries together.

You will use: requests, .json(), a list, nested dictionaries, a loop

1. Ask the user for a country name.
2. Call `https://restcountries.com/v3.1/name/` followed by the country name, for example .../name/kenya.
3. The answer is a **list**. Use the first item: `country = data[0]`. Print country to see its structure.
4. Pick out: the common name, the capital, the region and the population. Use f-strings with a thousands separator, for example `{population:,}`.
5. Loop through the languages and print them.
6. If the response status is not 200, print "Country not found" and ask again.

```
==== Kenya ====
Capital:    Nairobi
Region:     Africa
Population: 53,771,300
Languages:  English, Swahili
```

Hints (try without them first)
country["name"]["common"] is a dictionary inside a dictionary. country["capital"] is a list, so use [0]. response.status_code tells you if the request worked.

UNGUIDED 4.3 City Comparison Tool

Brief only. No steps, no hints, no starter code.

Brief
Ask the user for two city names. Use the geocoding API (`https://geocoding-api.open-meteo.com/v1/search?name=CITY&count=1`) to find each city's latitude and longitude, then use the weather API from Guided 4.1 to get each city's temperature. Print which city is warmer and by how many degrees. Keep the program running until the user types "quit".

Must include: at least 3 functions, try / except for failed requests, a loop, and a friendly message when a city is not found.

Extension: also show the wind speed and say which city is windier, or compare 3 cities and print a ranked list.

Explain it (be ready to answer these)
What does .json() do? What is the difference between a list and a dictionary in the response? What happens if the internet is off?

STAGE 5

Capstone Project

Choose **one** of the three projects below, or propose your own of the same size. Plan it, build it in phases, and test after each phase. No steps are given: you make the design decisions.

Option	Brief
A. School Records Manager	A menu-driven program to add students, record marks and fee payments, search, and show a class report (average, top student, passes, fees still owed).
B. Country Explorer	A menu-driven program that looks up countries from an API, lets you save favourites, compare the population of two countries, and show your favourites list.
C. Personal Finance Manager	A menu-driven program to record income and expenses by category, show monthly totals and a savings rate, and warn when spending is too high.

Build it in phases

Phase	What to do
1. Plan	Write what the program must do in plain sentences. Decide what data you will store and how.
2. Skeleton	Create the menu loop and one empty function for each option. Run it.
3. Core	Build the main features one at a time. Run after each one.
4. Protect	Handle wrong input, empty data and failed requests so nothing crashes.
5. Polish	Tidy the output. Use clear names. Add a comment above each function.
6. Present	Run it for someone and explain how it works, in your own words.

Requirements

- At least **8 functions** and a main() function.
- Data stored in a list of dictionaries (or dictionaries of lists).
- Input validation: typing letters where a number is expected must not crash the program.
- Option B must use an API. For A and C, an API is a bonus.

Stretch goals

- Save your data to a .json file on exit and load it on start-up.
- Edit and delete records.
- Add a simple login before the menu.
- Show a ranked or sorted report.

Self-check before you finish

Check	Done
Every menu option works, and Exit closes the program properly.	
My program is split into functions that each do one job.	
Wrong input never crashes the program.	
My output is neat and labelled.	
I tested after every phase.	
I can explain every part of my code in my own words.	

My Build Log

After each project, write one line. Over time this page becomes proof of what you can build.

Project	Date	What I built / what I learned	What I would improve